Operations Research

Totally Unimodular Matrix

No rights reserved, feel free to distribute.

# Approximation Algorithms for the Traveling Salesman Problem

A Chapter from Project Studies (2026)

PenguinPuffs (Prajeet Pushkar)

# Chapter 1

# Approximation Algorithms

## 1.1 Introduction

We know that the traveling salesman problem is NP-hard [14]. Thus, unless **P**=**NP**, there is no polynomial-time algorithm that always finds an optimal tour. One way to cope with this is to relax the running-time requirement and allow algorithms whose worst-case running time is super-polynomial, such as dynamic programming or branch-and-cut methods. Alternatively, one may relax the quality requirement and accept heuristic solutions, which often run very quickly and perform extremely well in practice (e.g., the Lin-Kernighan heuristic), but provide no worst-case performance guarantee. A third approach is to require guarantees on both running time and solution quality but not require optimality. This is essentially an approximation algorithm. An $\alpha$-approximation algorithm for an optimization problem is a polynomial-time algorithm that for all instances of the problem produces a solution whose value is within a factor of $\alpha$ of the value of an optimal solution.

In its full generality, the traveling salesman problem is not approximable within any constant factor unless **P**=**NP** [24, Theorem 3.6]. Consequently, meaningful approximation guarantees require additional structural assumptions. A particularly important special case is the *metric* TSP, where the edge costs are symmetric and satisfy the triangle inequality. Although this variant remains NP-hard, it admits constant-factor approximation algorithms. Most notably, Christofides [7] presented a 3/2-approximation algorithm, one of the most celebrated results in the field of approximation algorithms. For the asymmetric case, Frieze et al. [10] provided the first $O(\log n)$ factor approximation. This factor was subsequently improved to $0.999n$ [3], $0.842n$ [13] and $0.666n$ [9]. A significant breakthrough by Asadpour et al. [1] improved this factor to $O(\log n/\log\log n)$. The first constant-factor approximation for ATSP was due to Svensson, Tarnawski, and Végh [21], who proved a bound of 506 relative to the Held–Karp relaxation. This constant was subsequently improved by Traub and Vygen [22] to a factor of $22+\varepsilon$, and later to $17+\varepsilon$ in their monograph [23].

Going back to the TSP formulation of Dantzig, Fulkerson, and Johnson [8], the linear programming relaxation obtained by replacing the integrality constraints with $0 \le x_{i,j} \le 1$ is commonly referred to as the *subtour elimination LP*, or simply the *subtour LP*. This relaxation is also known as the *Held–Karp relaxation*. It provides a polynomial-time computable lower bound on the optimal tour length and plays a central role in both exact and approximation algorithms for TSP.

In this chapter, we first study two approximation algorithms for the metric TSP, including the Christofides algorithm. We then implement several heuristics for the ATSP, including the repeated assignment heuristic, which provides an $O(\log n)$-approximation [10]. Inspired by the algorithm of Asadpour et al. [1], we also develop a simple heuristic tailored to our application. Finally, we examine the performance of the $O(\log n/\log\log n)$-approximation algorithm on the last-mile data.

## 1.2 Metric Traveling Salesman Problem

A metric TSP is a special case of the symmetric TSP in which the edge costs satisfy the triangle inequality, i.e.,

$$c(u, v) \leq c(u, w) + c(w, v)$$

for all vertices $u, v, w$ in the graph $G$.

### 1.2.1 Minimum Spanning Trees

Since the approximation algorithms presented in this section rely on minimum spanning trees, we briefly review their definition and basic properties.

**Definition 1.** Let $G = (V, E)$ be a connected undirected graph with edge weights $c : E \to \mathbb{R}$. A *minimum spanning tree* (MST) is a spanning tree $T \subseteq E$ that connects all vertices in $V$ and minimizes the total weight $\sum_{e \in T} c(e)$.

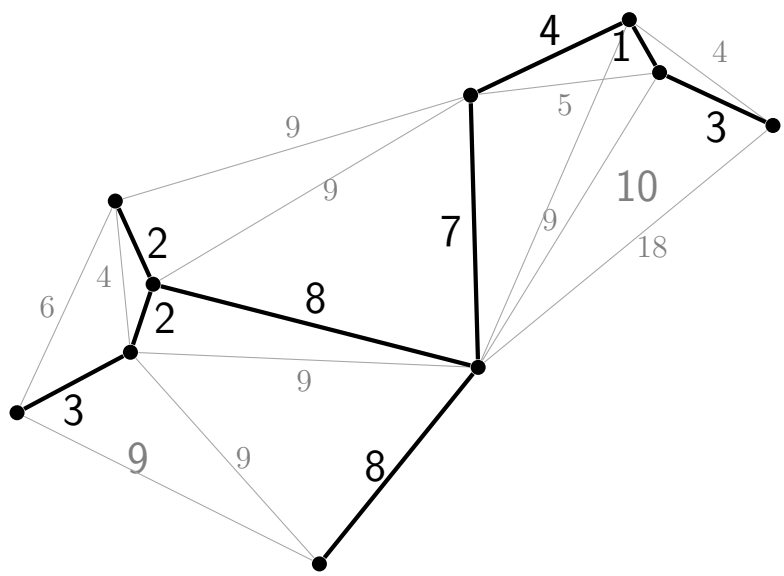



Figure 1.1: The minimum spanning tree for a planar graph.

It is well known that a minimum spanning tree can be computed in polynomial time, for instance using Prim's or Kruskal's algorithm. In contrast, the metric TSP remains NP-hard. Consequently, although an MST connects all vertices at minimum total cost among trees, it does not correspond to a Hamiltonian tour, nor does it necessarily approximate the optimal tour arbitrarily well. Nevertheless, the cost of a minimum spanning tree provides a natural lower bound on the cost of an optimal TSP tour. Indeed, removing any edge from a Hamiltonian cycle yields a spanning tree, implying that

$$c(\text{MST}) \leq c(\text{OPT}). \tag{1.1}$$

This observation forms the basis of the classical 2-approximation algorithm, which we present in the following section, followed by the 3/2-approximation algorithm of Christofides.

A related problem is the (metric) Steiner tree problem, which seeks a minimum-weight tree that spans a given subset of vertices (called terminals). Like the metric TSP, this problem admits a 2-approximation algorithm based on minimum spanning trees [24].

### 1.2.2 A simple factor 2 algorithm

This algorithm is commonly referred to as the *double-tree algorithm*.

1. Compute a minimum spanning tree $T$ of $G$.
2. Double every edge of $T$ to obtain an Eulerian multigraph.
3. Compute an Eulerian tour $\mathcal{T}$ of this multigraph.
4. Traverse the vertices in the order of their first appearance in $\mathcal{T}$, shortcutting repeated vertices, and output the resulting Hamiltonian cycle $\mathcal{C}$.

**Theorem 1.** The double-tree algorithm is a 2-approximation algorithm for the metric TSP.

*Proof.* Let $c(T)$ denote the cost of the minimum spanning tree. Since removing any edge from an optimal Hamiltonian cycle yields a spanning tree, we have $c(T) \leq c(\mathrm{OPT})$. After doubling every edge of $T$, the resulting Eulerian multigraph has total cost, i.e. $c(\mathcal{T}) = 2\,c(T)$. Hence, we have $c(\mathcal{T}) \leq 2\,c(\mathrm{OPT})$. Finally, by the triangle inequality, shortcutting repeated vertices in the Eulerian tour does not increase the total cost. Therefore, $c(\mathcal{C}) \leq c(\mathcal{T}) \leq 2\,c(\mathrm{OPT})$, which proves the claim. □

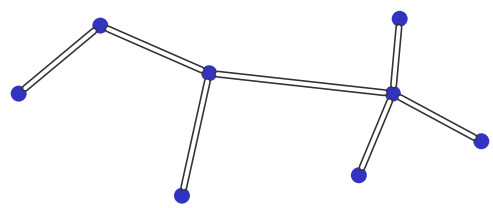


Figure 1.2: Illustration of the double-tree algorithm [23]: the edges of the minimum spanning tree are doubled to obtain an Eulerian multigraph.

Before we dive into Christofides algorithm, we briefly review perfect matchings.

### 1.2.3 Perfect Matchings

**Definition 2.** Let $G = (V, E)$ be a graph, a *matching* $M \subseteq E$ is a set of disjoint edges such that at most one edge in $M$ is incident to any node in $V$. A matching is *perfect* if it covers all nodes. The *weight* of a matching is the sum of the weights of the edges in the matching.

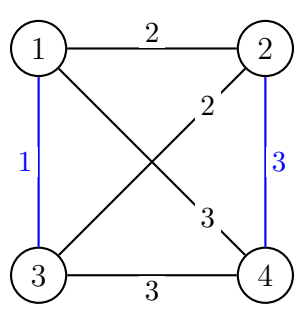



Figure 1.3: A minimum weight perfect matching in $K_4$, the number of perfect matchings is 3

### 1.2.4 Christofides Algorithm

1. Compute a minimum spanning tree $T$ of $G$.
2. Let $V' \subseteq V$ be the set of vertices of odd degree in $T$. Compute a minimum-cost perfect matching $M$ on $V'$ and add $M$ to $T$. The resulting multigraph is Eulerian.
3. Compute an Eulerian tour $\mathcal{T}$ of this multigraph.
4. Shortcut repeated vertices in $\mathcal{T}$ and output the resulting Hamiltonian cycle $\mathcal{C}$.

**Theorem 2.** Christofides algorithm is a $\frac{3}{2}$-approximation algorithm for the metric TSP.

*Proof.* Let $H$ be an optimal TSP tour and denote its cost by $c(\mathrm{OPT})$. As before, removing any edge from $H$ yields a spanning tree, so $c(T) \leq c(\mathrm{OPT})$. Let $V'$ be the set of vertices of odd degree in $T$. Since every graph contains an even number of odd-degree vertices, $|V'|$ is even, and a perfect matching on $V'$ exists. Consider the tour $H$ and restrict it to the vertices in $V'$ by shortcutting vertices not in $V'$. By the triangle inequality, the resulting tour $H'$ satisfies $c(H') \leq c(\mathrm{OPT})$. Since $|V'|$ is even, the edges of $H'$ can be partitioned into two perfect matchings $M_1$ and $M_2$, obtained by taking alternating edges along the cycle. Thus,

$$c(M_1) + c(M_2) = c(H').$$

Let $M$ be a minimum-cost perfect matching on $V'$. Then

$$c(M) \leq \min\{c(M_1), c(M_2)\} \leq \frac{1}{2}c(H') \leq \frac{1}{2}c(\mathrm{OPT}).$$

The multigraph obtained by adding $M$ to $T$ is Eulerian, and hence its Eulerian tour $\mathcal{T}$ has cost

$$c(\mathcal{T}) = c(T) + c(M).$$

Finally, shortcutting repeated vertices does not increase the cost by the triangle inequality, so

$$c(\mathcal{C}) \leq c(\mathcal{T}) \leq c(T) + c(M) \leq c(\mathrm{OPT}) + \frac{1}{2}c(\mathrm{OPT}) = \frac{3}{2}c(\mathrm{OPT}),$$

which completes the proof. □

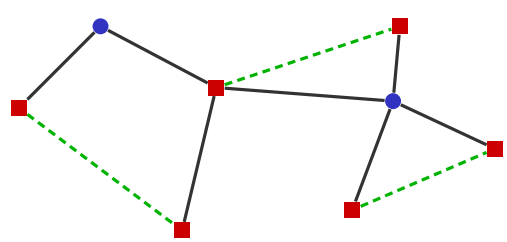


Figure 1.4: Illustration of the Christofides algorithm [23]: instead of doubling all edges, a minimum-cost perfect matching is added on the odd-degree vertices (shown in red).

The spanning tree is bounded by $c(T) \leq c(\mathrm{OPT})$, while the matching cost is bounded by decomposing the optimal Hamiltonian cycle into two perfect matchings on the odd-degree vertices. Combining these bounds yields the 3/2-approximation guarantee.

## 1.3 Repeated Assignment Heuristic

The first nontrivial approximation algorithm for the ATSP was developed by Frieze, Galbiati, and Maffioli [10], achieving an $O(\log n)$ approximation ratio. This algorithm, commonly known as the *cycle cover algorithm* or *repeated assignment heuristic*, constructs a Hamiltonian cycle through iterative application of minimum-cost assignment problems. A cycle cover of a graph $G = (V, A)$ is a subset $F \subseteq A$ of arcs such that every node has in-degree and out-degree equal to one (i.e., Eulerian) but not necessarily connected. A connected cycle cover would be a Hamiltonian tour and the optimal tour cost would be that of the optimal solution to the assignment problem (AP).

### 1.3.1 Assignment Problem

Consider a directed graph $G := (V \cup \{0\}, A)$ where $\{0\}$ represents the depot node and $V := \{1, \ldots, n\}$ is the node set. The arc set $A := \{(i, j) \in (V \cup \{0\}) \times (V \cup \{0\}) \mid i \neq j,\ c_{ij} < \infty\}$ contains all available arcs with finite cost $c_{ij} > 0$. It is assumed that the costs satisfy the asymmetric triangle inequality on available arcs, i.e., for any $i, j, k \in V \cup \{0\}$ with $(i, j), (j, k), (i, k) \in A$, $c_{ik} \leq c_{ij} + c_{jk}$. The assignment problem relaxation of the ATSP is formulated as follows:

$$\min \quad \sum_{(i,j)\in A} c_{ij} x_{ij} \tag{1.2}$$

$$\text{s.t.} \quad \sum_{j:(i,j)\in A} x_{ij} = 1 \qquad \forall i \in V \cup \{0\}, \tag{1.3}$$

$$\sum_{i:(i,j)\in A} x_{ij} = 1 \qquad \forall j \in V \cup \{0\}, \tag{1.4}$$

$$x_{ij} \in \{0, 1\} \qquad \forall (i, j) \in A. \tag{1.5}$$

This formulation enforces exactly one outgoing and one incoming arc at each node, which is a necessary but not sufficient condition for a Hamiltonian cycle. The solution is a cycle cover, i.e., a collection of directed disjoint cycles.

**Lemma 1.** Let OPT denote the optimal cost of ATSP and $\mathrm{OPT}_{\mathrm{AP}}$ the optimal cost of the assignment problem (or minimum-cost cycle cover). Then $\mathrm{OPT}_{\mathrm{AP}} \leq \mathrm{OPT}$.

*Proof.* Every Hamiltonian cycle is a feasible solution to the assignment problem. Therefore the assignment problem is a relaxation of ATSP and its optimal cost cannot exceed that of ATSP. □

### 1.3.2 Algorithm

1. Compute a minimum-cost cycle cover with cycles $C_1, \ldots, C_k$ on the node set $V \cup \{0\}$ using the assignment relaxation.
2. From each $C_i$, select an arbitrary proxy node $v_i$.
3. Recursively apply the same procedure on the reduced graph containing only the proxy nodes, $G[\{v_1, \ldots, v_k\}]$ to obtain a tour $C$.
4. Combine this with the cycle cover, $C' = C \cup C_1 \cup C_2 \ldots C_k$, to obtain an Eulerian graph $C'$.
5. Repeated nodes in $C'$ are shortcut, using the triangle inequality to obtain a Hamiltonian cycle $\mathcal{C}$ on $V \cup \{0\}$ that starts and ends at the depot.

**Lemma 2.** Given $G$ satisfies the asymmetric triangle inequality. Then for any $S \subseteq V \cup \{0\}$, the cost of an optimal TSP tour in $G[S]$ is at most the cost of an optimal TSP tour in $G$.

*Proof.* Let $C$ be an optimal ATSP tour in $G$. Consider the order in which the nodes of $S$ appear along $C$. Since $G$ satisfies the triangle inequality, $C$ can be shortcut to produce another cycle $C'$ with only $S$ nodes and whose cost is at most the cost of $C$. □

**Theorem 3.** The repeated assignment heuristic is a $\log_2 n$-approximation algorithm for ATSP.

*Proof.* We prove the theorem by induction on $N = |V \cup \{0\}|$, the number of nodes in $G$. If $N \leq 2$, the instance is trivial and the algorithm returns an optimal tour. For $N > 2$, the number of cycles $k$ in the cycle cover is at most $\lfloor N/2 \rfloor$, since each cycle in the cover has at least 2 nodes and they are disjoint. Thus $k \leq \lfloor N/2 \rfloor$. Let $C_1, \ldots, C_k$ be the cycles on $V \cup \{0\}$ obtained after solving the assignment relaxation. Let $S = \{v_1, \ldots, v_k\}$ be the set of proxy nodes, containing one node arbitrarily selected from each cycle, so $|S| = k \leq \lfloor N/2 \rfloor < N$. Let $\mathrm{OPT}(S)$ be the cost of an optimal tour in $G[S]$. From Lemma 2, we have $\mathrm{OPT}(S) \leq \mathrm{OPT}$. The algorithm recurses on the proxy nodes $S$. Since $|S| < N$, by the inductive hypothesis, the cost of the tour $C$ returned by the recursive call is

$$\mathrm{cost}(C) \leq (\log_2 |S|) \cdot \mathrm{OPT}(S) \leq (\log_2 |S|) \cdot \mathrm{OPT}.$$

Combining $C$ with the original cycle cover gives an Eulerian graph $C' := C \cup C_1 \cup \ldots C_k$, whose cost satisfies

$$\mathrm{cost}(C') \leq \mathrm{cost}(C) + \sum_{i=1}^{k} \mathrm{cost}(C_i) \leq \log_2 |S| \cdot \mathrm{OPT} + \mathrm{OPT},$$

because the cycle cover cost (i.e., $\mathrm{OPT}_{\mathrm{AP}}$) is at most OPT (Lemma 1). Finally, since shortcutting does not increase the cost and $|S| \leq \lfloor N/2 \rfloor$, we have

$$\mathrm{cost}(\mathcal{C}) \leq \log_2 |S| \cdot \mathrm{OPT} + \mathrm{OPT} \leq \log_2(N/2) \cdot \mathrm{OPT} + \mathrm{OPT} = (\log_2 N) \cdot \mathrm{OPT},$$

Intuitively, at each recursive step the number of nodes decreases by at least a factor of two, since each cycle contains at least two nodes. Therefore, the number of recursion levels is at most $\log_2 N$, and at each level we incur a cost of at most OPT from the cycle cover. □

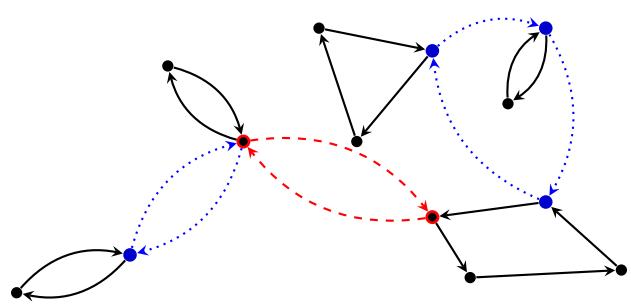


Figure 1.5: Illustration of the repeated assignment heuristic [23]. The first iteration chooses a minimum-cost cycle cover. The second iteration chooses a proxy node from each connected component and adds a minimum-cost cycle cover on these (blue). After two more arcs are added in the third iteration (red), the digraph is connected, and the algorithm terminates.

This heuristic can be viewed as iteratively enforcing connectivity on the assignment relaxation without explicitly adding subtour elimination constraints. This approximation factor factor was subsequently improved to $0.999 \log_2 n$ [3], $0.842 \log_2 n$ [13] and $0.666 \log_2 n$.[9].

We now discuss the $O(\log n/\log\log n)$-approximation by Asadpour et al. [1], which was a major breakthrough result in this field. In contrast to the repeated assignment heuristic, which relies on the assignment relaxation, this algorithm builds upon the Held–Karp relaxation [12] of ATSP.

The algorithm proceeds in three main stages. First, an optimal solution to the Held–Karp linear programming relaxation is computed. From this fractional solution, a probability distribution over spanning trees of the underlying undirected graph is constructed. A spanning tree is then sampled from this distribution such that it is *thin* with respect to the fractional solution, meaning that its cut structure approximately preserves the LP values. Second, the sampled tree is oriented and augmented to obtain a strongly connected Eulerian multigraph. This augmentation is achieved by solving a minimum-cost circulation problem that balances the in-degree and out-degree at each node. Finally, an Eulerian tour of the resulting multigraph is computed and shortcut using the triangle inequality to obtain a Hamiltonian cycle.

## 1.4 Held–Karp Relaxation

In this section, we introduce the Held–Karp relaxation [12], also known as the subtour LP [8], which serves as a linear programming relaxation of the ATSP. Let $V_0 := V \cup \{0\}$ denote the full node set including the depot. The number of nodes in the full node set is, $|V_0| = n + 1$. Given a directed graph $G := (V_0, A)$ with $V_0$ as nodes, $A := \{(i,j) \in V_0 \times V_0 \mid i \neq j,\ c_{ij} < \infty\}$ as the arc set. The Held–Karp relaxation is the following linear program:

$$\min \quad \sum_{(u,v)\in A} c(u,v)\, x(u,v) \tag{1.6}$$

$$\text{s.t.} \quad \sum_{u\in S,\, v\in V_0\setminus S} x(u,v) \geq 1 \qquad \forall S \subsetneq V_0, \tag{1.7}$$

$$\sum_{v\in V_0} x(u,v) = \sum_{v\in V_0} x(v,u) = 1 \qquad \forall u \in V_0 \tag{1.8}$$

$$x(u,v) \geq 0 \qquad \forall (u,v) \in A. \tag{1.9}$$

Constraint (1.7) ensures that for every subset $S \subsetneq V_0$, there is at least one arc leaving $S$, enforcing global connectivity of the solution. Constraints (1.8) enforce degree balance at each node, ensuring exactly one incoming and one outgoing arc.

The incidence vector of any Hamiltonian cycle is a feasible solution to the above LP. Therefore, the solution of the above LP provides a *lower bound* on the cost of the optimal ATSP tour.

$$c(x^*) \leq \mathrm{OPT}. \tag{1.10}$$

It is well-known that an optimal solution $x^*$ to this relaxation can be computed in polynomial time [1]. Without loss of generality, we assume that $x^*$ is an extreme point of the corresponding polytope.

## 1.5 Thin Spanning Trees

Let $\vec{T}$ be an orientation of a spanning tree $T$ over the node set $V_0$. In order to obtain a feasible ATSP tour, the oriented tree must be augmented to become Eulerian, i.e., every node must have equal in-degree and out-degree and the graph must be strongly connected.

This problem can be formulated as a minimum-cost circulation (or flow) problem and solved efficiently. If $E_{\text{aug}}$ denotes the set of augmentation arcs, then the cost of the resulting ATSP tour after shortcutting the Eulerian multigraph is

$$c(\text{ATSP}) = c(\vec{T}) + c(E_{\text{aug}}). \tag{1.11}$$

Therefore, in order to upper-bound the approximation ratio of the algorithm, one must control both the cost of the sampled tree $\vec{T}$ and the cost of the Eulerian augmentation. The notion of *thinness* precisely captures the property that ensures that the augmentation cost remains small.

### 1.5.1 Symmetrization and the Spanning Tree Polytope

To construct thin spanning trees, we derive an undirected fractional structure from the optimal Held–Karp solution $x^*$. This is done by symmetrizing $x^*$ to obtain a point in the spanning tree polytope. Define the symmetrized vector $z^* \in \mathbb{R}^{\binom{V_0}{2}}$ by

$$z^*_{\{u,v\}} := \frac{n}{n+1}\left(x^*_{uv} + x^*_{vu}\right) \tag{1.12}$$

The spanning tree polytope $P$ of a graph $(V_0, E)$ is defined by the following inequalities [11]:

$$\begin{aligned} P = \{z \in \mathbb{R}^E : \sum_{e\in E} z_e &= |V_0| - 1, \\ \sum_{e\in E(S)} z_e &\le |S| - 1 && \forall S \subseteq V_0 \\ z_e &\ge 0 && \forall e \in E.\} \end{aligned} \tag{1.13}$$

Here, $E(S)$ denotes the set of edges with both head and tail in $S \subseteq V_0$. The relative interior of $P$ consists of points that satisfy all these inequalities strictly.

We now verify that the symmetrized vector $z^*$ satisfies these constraints (1.13). The equality

$$\sum_{e\in E} z^*_e = |V_0| - 1$$

follows directly from the scaling factor $n/(n+1)$ and the fact that $x^*$ has total degree 2 at each node. It remains to show that the cut constraints are satisfied strictly. For any nonempty subset $S \subset V_0$, we have

$$\begin{aligned} \sum_{e\in E(S)} z^*_e &= \frac{1}{2}\left(\sum_{v\in S}\sum_{u\in V_0} z^*_{\{u,v\}} - \sum_{\substack{u\in S\\ v\in V_0\setminus S}} z^*_{\{u,v\}}\right) \\ &= \frac{n}{2(n+1)}\left(\sum_{v\in S}\sum_{u\in V_0}\left(x^*_{uv} + x^*_{vu}\right) - \sum_{\substack{u\in S\\ v\in V_0\setminus S}}\left(x^*_{uv} + x^*_{vu}\right)\right). \end{aligned} \tag{1.14}$$

Using the degree constraints (1.8), we have

$$\sum_{v\in S}\sum_{u\in V_0}\left(x^*_{uv} + x^*_{vu}\right) = 2|S|. \tag{1.15}$$

Moreover, using the cut constraint (1.7), we have

$$\sum_{\substack{u\in S\\ v\in V_0\setminus S}} \left(x^*_{uv} + x^*_{vu}\right) \geq 2. \tag{1.16}$$

Substituting (1.15) and (1.16) into (1.14), we have

$$\begin{aligned}\sum_{e\in E(S)} z^*_e &\leq \frac{n}{2(n+1)}(2|S|-2)\\ &= \frac{n}{n+1}(|S|-1) < |S|-1. \end{aligned}\tag{1.17}$$

Thus, $z^*$ lies in the spanning tree polytope. We now formally define thin trees.

**Definition 3** (Thin trees [1])**.** Let $G = (V_0, E)$ be an undirected graph and let $z \in \mathbb{R}^E_{\geq 0}$ be a point in the spanning tree polytope. A spanning tree $T$ is called $\alpha$-thin with respect to $z$ if for every subset $S \subset V_0$,

$$|T \cap \delta(S)| \leq \alpha \cdot z(\delta(S)).$$

Here, $\delta(S)$ denotes the set of edges with exactly one endpoint in $S$. We say that $T$ is $(\alpha, s)$-thin if it is $\alpha$-thin and additionally $c(T) \leq s \cdot c(x^*)$.

Definition 3 disregards the orientation of arcs and considers the underlying undirected graph. This allows the algorithm to work with the spanning tree polytope, while the orientation is restored later when the tree is augmented to obtain an Eulerian directed multigraph. Thin trees provide the key link between the fractional Held–Karp solution and an integral ATSP tour. The following result shows that constructing a thin tree with bounded cost directly yields a good approximation.

**Theorem 4** (Upper bound [1])**.** Let $x^*$ be an optimal solution to the Held-Karp relaxation, and $z^*$ be the symmetrized and scaled down version of $x^*$ (1.12) in the spanning tree polytope. If $T^*$ is an $(\alpha, s)$-thin spanning tree with respect to $z^*$ for some $\alpha$ and $s$, then we can find in polynomial-time a Hamiltonian cycle whose cost is at most $(2\alpha + s)\, c(x^*) \leq (2\alpha + s)\,\mathrm{OPT}$.

## 1.6 Maximum Entropy Sampling

A natural way to round a fractional vector $x = (x_1, \ldots, x_n)$ with $0 \leq x_i \leq 1$, for each $i$, is to perform *independent randomized rounding* [20], i.e., each variable $x_i$ is set to 1 with probability $x_i$ and to 0 with the remaining probability. This procedure preserves the *marginals*, since $\mathbb{E}[\hat{x}_i] = x_i$. However, independent rounding ignores the combinatorial structure of the solution. In particular, when applied to the fractional spanning-tree vector $z^*$ it may produce a disconnected graph or directed cycles, thus violating the required tree structure. To avoid this issue we seek a probability distribution over spanning trees that:

- preserves the edge marginals given by $z^*$, and
- exhibits negative correlation properties [19].

Negative correlation is important because it allows the use of Chernoff-type concentration bounds, which are crucial in proving that the sampled tree is thin with high probability.

Among all distributions over spanning trees that preserve the desired marginals, we consider the one with maximum entropy. Intuitively, this is the "most random" distribution consistent with the given marginals. It is well known that maximum entropy distributions subject to linear constraints are exponential distributions [5]. In particular, any set of edge weights $\gamma_e$ defines an exponential distribution over spanning trees of the form

$$p(T) \propto \exp\left(\sum_{e \in T} \gamma_e\right),$$

for some edge weights $\gamma_e$. Moreover, any such exponential distribution is the maximum entropy distribution corresponding to its induced marginals.

Let $E$ denote the support graph of $z^*$ obtained by disregarding the directions of the arcs. We aim to find weights $\{\tilde{\gamma}_e\}_{e \in E}$ such that the induced distribution

$$\tilde{p}(T) \propto \exp\left(\sum_{e \in T} \tilde{\gamma}_e\right), \tag{1.18}$$

approximately preserves the marginals imposed by $z^*$. That is, for every edge $e \in E$,

$$\sum_{T \in \mathcal{T} : e \in T} \tilde{p}(T) \leq (1 + \varepsilon)\, z_e^*, \tag{1.19}$$

for some sufficiently small $\varepsilon > 0$.

The problem of finding the maximum entropy distribution that preserves the marginals can be formulated as a convex optimization problem with exponentially many variables. This can be approximated efficiently using a combinatorial algorithm based on multiplicative weights [1].

The goal of this procedure is to produce a spanning tree that is sufficiently thin with respect to the vector $z^*$. Preserving the marginals induced by $z^*$ is crucial for proving the thinness guarantees of the approximation algorithm. Thus, representing $z^*$ as a probability distribution over spanning trees via a maximum-entropy distribution provides a natural way to obtain the desired random tree.

### 1.6.1 Maximum Entropy Distribution

Let $\mathcal{T}$ denote the set of all spanning trees of $G = (V_0, E)$ and let $z$ be a point in the spanning tree polytope $P$. We recall that the entropy of a discrete probability distribution $p$ over $\mathcal{T}$ is defined as

$$H(p) = -\sum_{T \in \mathcal{T}} p(T) \log p(T). \tag{1.20}$$

The maximum entropy distribution with marginals $z$ can be obtained by solving the convex program:

$$\inf \quad \sum_{T \in \mathcal{T}} p(T) \log p(T) \tag{1.21}$$

$$\text{s.t.} \quad \sum_{T \ni e} p(T) = z_e \qquad \forall e \in E \tag{1.22}$$

$$\sum_{T \in \mathcal{T}} p(T) = 1 \tag{1.23}$$

$$p(T) \geq 0 \qquad \forall T \in \mathcal{T}. \tag{1.24}$$

The objective function is strictly convex and the feasible region is compact, therefore the convex program has a unique optimal solution $p^*(\cdot)$. The value $p^*(T)$ determines the probability of sampling any tree $T$ in the maximum entropy rounding scheme.

For every $e \in E$, a Lagrange multiplier $\delta_e$ is associated to the constraint corresponding to the marginal $z_e$ and the Lagrange function is defined as

$$\begin{aligned} L(p,\delta) &= \sum_{T\in\mathcal{T}} p(T)\log p(T) - \sum_{e\in E}\delta_e\left(\sum_{T\ni e} p(T) - z_e\right) \\ &= \sum_{e\in E}\delta_e z_e + \sum_{T\in\mathcal{T}}\left(p(T)\log p(T) - p(T)\sum_{e\in T}\delta_e\right) \end{aligned} \tag{1.25}$$

Thus, the Lagrange dual to the convex program is $\sup_\delta \inf_{p\geq 0} L(p,\delta)$. For every $T \in \mathcal{T}$, $p(T)$ must minimize the convex function

$$p(T)\log p(T) - p(T)\delta(T),$$

where, $\delta(T) = \sum_{e\in T}\delta_e$. Taking partial derivatives with respect to $p(T)$, we get

$$p(T) = e^{\delta(T)-1} \tag{1.26}$$

Thus,

$$\inf_{p\geq 0} L(p,\delta) = \sum_{e\in E}\delta_e z_e - \sum_{T\in\mathcal{T}} e^{\delta(T)-1}$$

Changing variables $\gamma_e = \delta_e - \frac{1}{n}$ for $e \in E$, the Lagrange dual becomes,

$$\sup_\gamma\left(1 + \sum_{e\in E} z_e\gamma_e - \sum_{T\in\mathcal{T}} e^{\gamma(T)}\right) \tag{1.27}$$

The assumption that $z$ is in the interior of the spanning tree polytope implies that strong duality holds. Also, $z$ can be written as a convex combination of all spanning trees in $\mathcal{T}$ such that the coefficient corresponding to each spanning tree is positive. This means that there is a point $p(.)$ in the interior of the convex program. So, the convex program satisfies the Slater's condition which is a sufficient condition for strong duality to hold for a convex optimization problem [5]. Therefore, if $\gamma^*$ is the optimal solution of (1.27),

$$p^*(T) = e^{\gamma^*(T)} \tag{1.28}$$

Thus, solving the Lagrange dual problem yields edge weights $\gamma^*$ that define an exponential distribution over spanning trees. Sampling from this distribution produces trees whose marginals approximately match $z$.

### 1.6.2 Sampling a $\lambda$-Random Tree

Given edge weights $\lambda_e \geq 0$ for $e \in E$, a *$\lambda$-random spanning tree* of $G$ is a spanning tree $T$ chosen with probability

$$\Pr[T] \propto \prod_{e\in T}\lambda_e.$$

In particular, setting $\lambda_e = e^{\gamma_e}$ shows that sampling from the exponential family distribution

$$p(T) \propto \exp\left(\sum_{e\in T}\gamma_e\right)$$

is equivalent to sampling a $\lambda$-random spanning tree [1, 15].

**Sequential Sampling Procedure**

One practical method to sample a $\lambda$-random tree works as follows [15]:

1. Order the edges $e_1, \ldots, e_m$ of $G$ arbitrarily.
2. Process each edge sequentially. After the $j$-th edge $e_j$:
   - Decide probabilistically whether to include $e_j$ in the final tree.
   - The inclusion probability $p_j$ equals the probability that $e_j$ is in a $\lambda$-random tree conditioned on the choices pmade for $e_1, \ldots, e_{j-1}$.
3. Continue until all edges are processed; the resulting tree $T$ is a $\lambda$-random tree.

The probability $p_j$ can be computed efficiently by considering a graph $G'$ obtained from $G$ by contracting edges already included and deleting edges already excluded. Therefore, to obtain each $p_j$, we need to compute the probability $p_{G'}[\lambda, f]$ that some edge $f$ is in a $\lambda$-random tree for a given (multi)graph $G'$. For this computation, we use the Kirchhoff's matrix tree theorem [4, pp. 50–60].

**Computing Probabilities via Kirchhoff's matrix tree theorem**

Kirchhoff's matrix tree theorem states that $\sum_{T \in \mathcal{T}} \prod_{e \in T} \lambda_e$ for any graph $G$ is equal to the value of any cofactor of the weighted Laplacian $L$ where

$$L_{ij} = \begin{cases} -\lambda_e & e = (i,j) \in E \\ \sum_{e \in \delta(i)} \lambda_e & i = j \\ 0 & \text{otherwise.} \end{cases}$$

Using this result, all conditional probabilities $p_j$ can be computed in polynomial time.

## 1.7 Multiplicative Weights Algorithm

This combinatorial m efficiently computes edge weights $\tilde{\gamma}$'s that approximately preserve the marginal probabilities given by $z$ and solve the maximum entropy convex program.

1. Initialize $\gamma = \vec{0}$
2. While there exists an edge $e \in E$ with $q_e(\gamma) > (1+\varepsilon) z_e$:
   (a) Compute $\delta$ such that defining $\gamma'_e := \gamma_e - \delta, \quad \gamma'_f := \gamma_f \ \ \forall f \in E \setminus \{e\}$, ensures $q_e(\gamma') = (1+\varepsilon/2) z_e$.
   (b) Update $\gamma \leftarrow \gamma'$.
3. Output $\tilde{\gamma} := \gamma$

Here, $q_e(\gamma)$ is the probability that edge $e$ is included in a spanning tree under the current weights

$$q_e(\gamma) := \frac{\sum_{T \ni e} \exp(\gamma(T))}{\sum_T \exp(\gamma(T))}, \quad \text{where } \gamma(T) := \sum_{f \in T} \gamma_f,$$

Asadpour et al. [1] showed that the multiplicative weights algorithm converges in polynomial time and executes at most

$$O\left(\frac{1}{\varepsilon}|E|^2[|V|\log(|V|) - \log(\varepsilon z_{\min})]\right)$$

iterations, where $z_{\min} = 2^{-O(n \log n)}$.

## 1.8 ATSP heuristics

In this section, we discuss a novel heuristic, which is essentially based on approximation algorithms proposed in the literature [1, 7], and also discuss some classical heuristics [10].

### 1.8.1 A simple heuristic

This heuristic is inspired by the approximation algorithms of Asadpour et al. [1] and Christofides [7]. The main idea is to use the fractional solution of the Held–Karp relaxation to guide the construction of spanning trees that are likely to contain edges of a good tour. Held–Karp solution often assigns high fractional values to edges that appear in optimal or near-optimal tours. Biasing the tree construction toward these edges therefore increases the likelihood of obtaining a high-quality tour. To our knowledge, there is no formal worst-case guarantee for this heuristic.

1. We first solve the Held–Karp relaxation and obtain an optimal fractional solution $x^*$ together with the LP lower bound.

2. From $x^*$ we derive undirected edge weights by symmetrization,

$$w_{ij} = x^*_{ij} + x^*_{ji}$$

   Edges with larger $w_{ij}$ are interpreted as being more likely to appear in a good tour.

3. We then sample a number of spanning trees (i.e., $2\lceil \log n \rceil$) using a biased version of Kruskal's algorithm. The edge costs are perturbed according to the LP weights,

$$c'_{ij} = c_{ij}(1 - \alpha w_{ij}),$$

   where $\alpha \in [0, 1]$ controls the strength of the bias toward edges with high LP support. In our experiments we use $\alpha = 0.5$, which provides a moderate preference for such edges while still preserving the original cost structure.

4. For each sampled tree we perform the following steps:

   (a) Orient the edges to obtain a directed graph.

   (b) Compute the degree imbalance at each vertex.

   (c) Add a minimum-cost set of arcs to balance the degrees, producing an Eulerian directed multigraph.

   (d) Compute an Eulerian tour and apply shortcutting (using the triangle inequality) to obtain a Hamiltonian tour.

5. Among all sampled trees we return the tour with the minimum cost.

### 1.8.2 Classical heuristics

We briefly review some classical heuristics for (A)TSP. Their worst-case performance guarantees have been studied by Frieze et al. [10]. For the sake of notation in this section, we consider a digraph $G = (V, A)$ where $V$ contains all the nodes including the depot. Given a heuristic method $H$, its worst-case ratio $R_n(H)$ for $n$ node problems is given by

$$R_n(H) = \sup \frac{d(T)}{d(T^*)}, \quad d(T^*) > 0.$$

Here, $T^*$ is a minimum length tour and $T$ is a tour generated by heuristic $H$.

**Greedy algorithm** We preprocess arcs in non-decreasing order of cost. At each step, an arc $(i, j)$ is selected provided that (i) node $i$ has not yet been assigned an outgoing arc, (ii) node $j$ has not yet been assigned an incoming arc, and (iii) the addition of the arc does not form a cycle unless it forms a Hamiltonian cycle. For this heuristic, $R_n = n/2$.

**Nearest neighbor algorithm** Starting from any arbitrary node (e.g., depot), the algorithm repeatedly moves to the unvisited node that can be reached with minimum cost from the current node. This process continues until all nodes have been visited, after which the cycle is completed by returning to the starting node. For this heuristic, $R_n = n/2$.

**Cheapest insertion algorithm** Initially, the tour consists of only one node. At a general stage the algorithm maintains a subtour $T$ visiting a set of nodes $C \subseteq V$. A node $k \notin C$ is inserted between two consecutive nodes $i, j \in T$ so as to minimize the increase in tour cost, $c_{ik} + c_{kj} - c_{ij}$. This procedure is repeated until all nodes have been inserted. For this heuristic, $R_n = \Omega(n)$.

## 1.9 Implementation Details

This section describes our implementation of the $O(\log n/\log\log n)$-approximation algorithm of Asadpour et al. [1]. We follow Oveis Gharan's lecture notes [18] for the theoretical background.

**Electrical flows and effective resistance.** Given a weighted undirected graph $G = (V, E)$ with edge weights $w : E \to \mathbb{R}_{>0}$, we interpret each edge as a resistor of conductance $w(e)$ (resistance $1/w(e)$). An *electrical flow* sending one unit from $s$ to $t$ is the unique flow satisfying Ohm's law: there exist vertex potentials $p : V \to \mathbb{R}$ such that for every edge $e = (u, v)$,

$$x(e) = w(e)\big(p(u) - p(v)\big).$$

Writing $L_G = B^\top W B$ for the weighted Laplacian (where $B$ is the incidence matrix and $W$ the diagonal conductance matrix), the potential vector satisfies $L_G p = b_{s,t}$, so $p = L_G^\dagger b_{s,t}$ where $L_G^\dagger$ is the Moore–Penrose pseudoinverse [18].

The *effective resistance* between $s$ and $t$ is the potential difference induced by one unit of flow:

$$R_{\text{eff}}(s, t) = b_{s,t}^\top L_G^\dagger b_{s,t} = L_{ss}^\dagger + L_{tt}^\dagger - 2L_{st}^\dagger.$$

The second equality follows by expanding $(e_s - e_t)^\top L^\dagger (e_s - e_t)$ [18]. In our implementation, the pseudoinverse is computed via eigendecomposition for numerical stability.

**Maximum entropy distribution.** We compute $\gamma_e$'s for each edge $e \in E$ using the multiplicative weights algorithm of Asadpour et al. [1]. At each iteration, we define weights $\lambda_e = e^{\gamma_e}$ and construct the corresponding weighted Laplacian matrix $L$. Using the connection between spanning tree distributions and electrical flows, the marginal probability that an edge $e = (i, j)$ appears in a $\lambda$-random spanning tree is given by

$$q_e \;=\; \lambda_e \cdot R_{\text{eff}}(i, j).$$

where $R_{\text{eff}}(i, j)$ is computed with respect to the Laplacian weighted by $\{\lambda_e\}$. Whenever a marginal $q_e$ exceeds $(1 + \varepsilon)z_e^*$, the parameter $\gamma_e$ is decreased by a fixed step $\delta = \ln(1 + \varepsilon/2)$ to reduce the probability of selecting this edge. The update is *one-sided*, i.e., $\gamma_e$ is only decreased when $q_e$ is too large, and never explicitly increased. The procedure iterates until all edge marginals approximately satisfy their target values.

**Sampling $\lambda$-random spanning trees.** With $\gamma_e$ fixed, spanning trees are sampled according to

$$\Pr(T) \propto \prod_{e \in T} \lambda_e.$$

Enumerating all spanning trees is infeasible, so we use stochastic procedures.

**Contraction-based sequential sampling** Edges are processed in random order. For each edge $e$ still present in the contracted graph, its inclusion probability $q_e$ is computed from the Laplacian pseudoinverse. The edge is added to the tree with probability $q_e$, and the graph is contracted; otherwise, it is discarded. If fewer than $n - 1$ edges are selected due to numerical issues, the partial tree is completed greedily using the remaining highest-weight edges.

**Gumbel–Kruskal** If the contraction sampler fails, we draw independent Gumbel variables $g_e$ for each edge and compute

$$s_e = \log \lambda_e + g_e.$$

A Kruskal-like algorithm processes edges in decreasing order of $s_e$, producing a spanning tree sampled exactly from the $\lambda$-random distribution [16].

**Multiple samples and tour selection.** We independently sample $k = 2\lceil \ln n \rceil$ trees [1] and construct tours from each. The tour with minimum total cost is returned as the final solution.

## 1.10 Computational Experiments

The approximation algorithms described in this chapter [1, 10], along with the heuristic described in Section 1.8.1 are implemented in Python. All experiments run on a laptop with a 13th Gen Intel(R) Core(TM) i7-1355U(12) 5Ghz processor running Arch Linux on linux 6.18.9-arch1-2 kernel. Table 1.1 summarizes the results obtained by running all the algorithms on the 500 sampled routes [17], and reports which algorithm performed the best, for how many instances, number of time windows violated, time-window feasible routes and the runtimes. Figure 1.6–1.7a compares the performance of all the algorithms based on the tour costs and computational time. Figure 1.7b shows that the multiplicative weights procedure (with $\varepsilon = 0.2$) achieves marginal convergence quality above 80% for most instances, which is necessary in proving the worst-case guarantee.

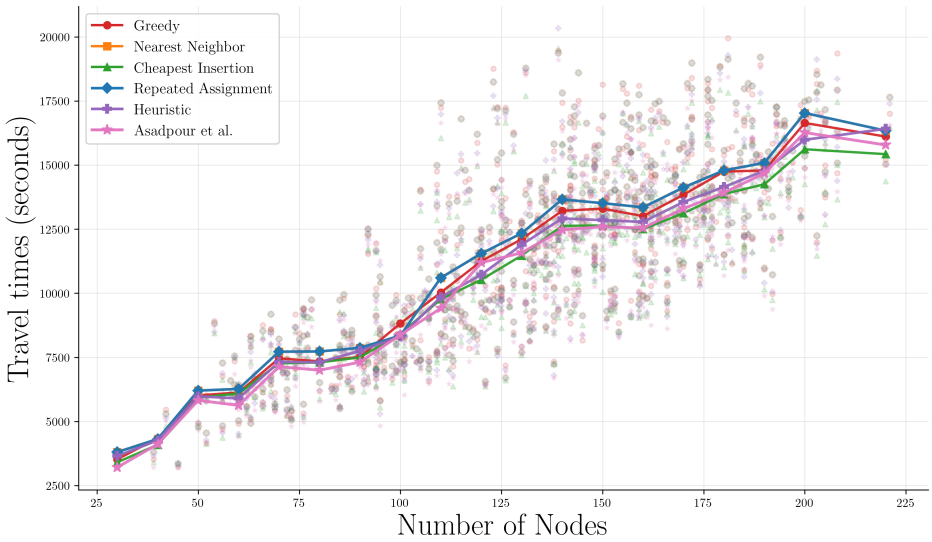
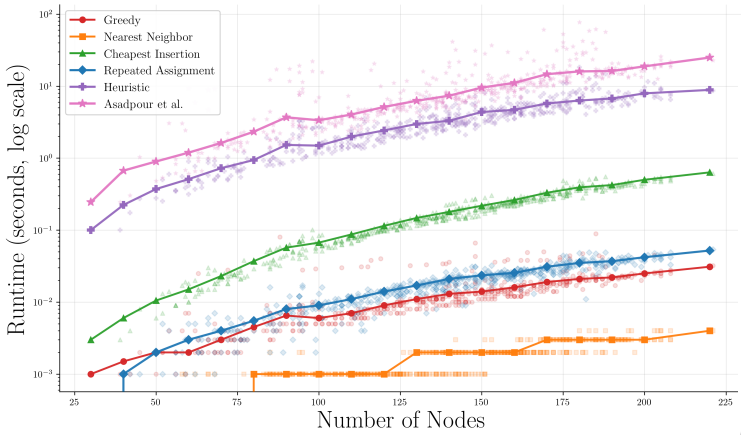
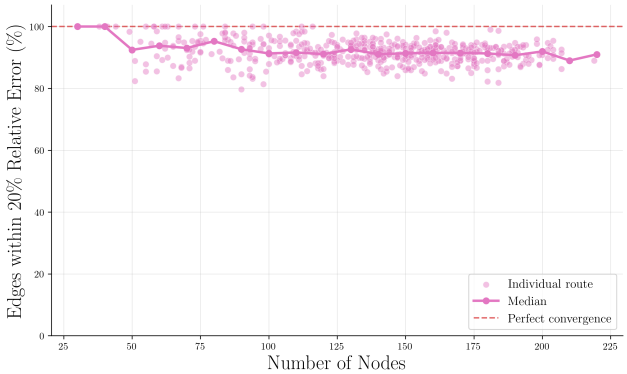


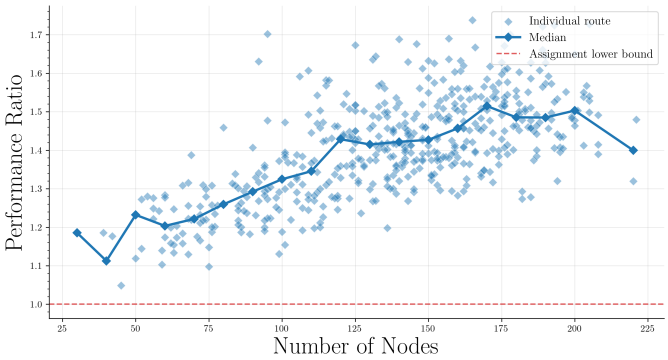

Figure 1.6: Travel times comparisons for all the algorithms covered in this chapter.



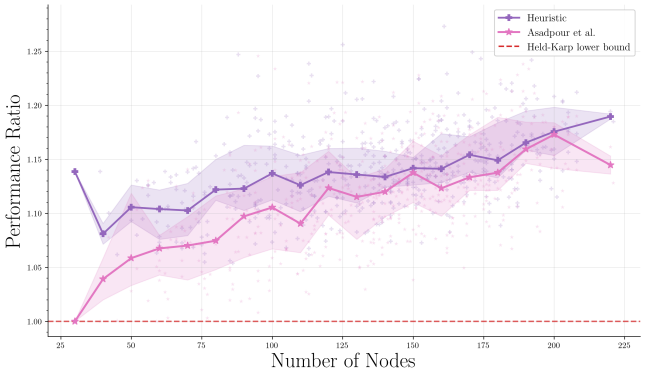

(a) Runtime comparisons of all the algorithms.




(b) Proportion of edges satisfying marginal convergence criterion $|q(e) - z^*(e)|/z^*(e) \leq 0.20$.




(c) Performance of the repeated assignment heuristic [10] with respect to the assignment relaxation lower bound.




(d) Performance comparison of heuristic 1.8.1 vs Asadpour et al. [1] approximation algorithm with respect to the Held–Karp relaxation lower bound.

Figure 1.7

| **Algorithm** | **Best Tours** | **TW Sat/Vio** | **TW Infeas.** | **Avg. Runtime (s)** |
|---|---|---|---|---|
| Cheapest Insertion | 240 | 5912/568 | 193 | 0.193 |
| Asadpour et al. | 208 | 5927/553 | 201 | 10.12 |
| Heuristic (Sec. 1.8.1) | 30 | 5916/564 | 191 | 3.75 |
| Greedy | 15 | 5610/870 | 268 | 0.014 |
| Nearest Neighbor | 7 | 5940/540 | 204 | 0.002 |
| Repeated Assignment | 0 | 5940/540 | 204 | 0.019 |

Table 1.1: Summary of computational results on 500 delivery routes.

Overall, the cheapest insertion and Asadpour et al. algorithms dominate in terms of solution quality, with cheapest insertion producing the largest number of best tours. Our heuristic provides a competitive trade-off between runtime and solution quality, outperforming simpler heuristics such as greedy and nearest neighbor. However, Figure 1.7d shows that our heuristic consistently produces worse solutions in comparison to the $O(\log n/\log\log n)$-approximation algorithm of Asadpour et al., although it achieves significantly lower runtimes. This highlights a clear trade-off between the running times and the solution quality.

| Algorithm | Mean | Median | Std | Min | Max | Lower bound |
|---|---|---|---|---|---|---|
| Heuristic (Sec. 1.8.1) | 1.1413 | 1.1384 | 0.0382 | 1.0538 | 1.2790 | Held–Karp relaxation |
| Asadpour et al. | 1.1186 | 1.1193 | 0.0484 | 1.0000 | 1.2455 | Held–Karp relaxation |
| Repeated Assignment | 1.4043 | 1.4051 | 0.1302 | 1.0486 | 1.7381 | Assignment relaxation |

Table 1.2: Performance ratios across algorithms.

## 1.11 Discussion

In this chapter, we studied various approximation algorithms proposed in the literature [1, 10] and implemented them for the asymmetric traveling salesman problem and compared the quality of solutions produced by them on 500 last-mile instances [17]. We also developed a simple heuristic, based on ideas of Christofides [7] and Asadpour et al. [1], making use of the optimal solution to the Held–Karp relaxation [12], which finds reasonably good solutions. Overall, all the algorithms provided much better solutions than their worst-case guarantees, highlighting the gap between theoretical worst-case analysis and empirical performance on structured instances such as last-mile instances. While time-window constraints were not considered in this chapter, future work could extend our heuristic to handle them, drawing inspiration from Chekuri and Korula [6], who study approximation algorithms for orienteering with time windows. Additionally, recent advances in approximation algorithms, e.g., Behrendt et al. [2], which generalizes the double tree and the Christofides algorithm for the asymmetric setting by parametrization, may provide valuable insights for further improving heuristic performance on asymmetric instances.

# Authorship

This chapter was written by Prajeet Pushkar.

# Bibliography


[1] Arash Asadpour, Michel X. Goemans, Aleksander Mkadry, Shayan Oveis Gharan, and Amin Saberi. An $O(\log n/\log\log n)$-approximation algorithm for the asymmetric traveling salesman problem. *Operations Research*, 65(4):1043–1061, 2017.

[2] Lukas Behrendt, Katrin Casel, Tobias Friedrich, J. A. Gregor Lagodzinski, Alexander Löser, and Marcus Wilhelm. From symmetry to asymmetry: Generalizing tsp approximations by parametrization, 2020.

[3] Markus Bläser. A new approximation algorithm for the asymmetric TSP with triangle inequality. In *Proceedings of the fourteenth annual ACM-SIAM Symposium on Discrete algorithms*, pages 638–645, 2002.

[4] Béla Bollobás. *Modern Graph Theory*, volume 184 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 2002.

[5] Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

[6] Chandra Chekuri and Nitish Korula. Approximation algorithms for orienteering with time windows. Technical Report 0711.4825, arXiv, 2007.

[7] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report Technical Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, Pittsburgh, PA, 1976.

[8] George B. Dantzig, Ray Fulkerson, and Selmer M. Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research*, 2(4):393–410, 1954.

[9] Uriel Feige and Mohit Singh. Improved approximation ratios for traveling salesperson tours and paths in directed graphs. In *International Workshop and International Workshop on Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*, 2007.

[10] Alan M. Frieze, Giulia Galbiati, and Francesco Maffioli. On the worst-case performance of some algorithms for the asymmetric traveling salesman problem. *Networks*, 12(1):23–39, 1982.

[11] Martin Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*, volume 2 of *Algorithms and Combinatorics*. Springer-Verlag, Berlin, Heidelberg, 1988.

[12] Michael Held and Richard M. Karp. The traveling-salesman problem and minimum spanning trees. *Operations Research*, 18(6):1138–1162, 1970.

[13] Haim Kaplan, Moshe Lewenstein, Nira Shafrir, and Maxim Sviridenko. Approximation algorithms for asymmetric tsp by decomposing directed regular multigraphs. *J. ACM*, 52(4):602–626, July 2005.

[14] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103, New York, 1972. Plenum Press.

[15] V.G Kulkarni. Generating random combinatorial objects. *Journal of Algorithms*, 11(2):185–207, 1990.

[16] Chris J. Maddison, Daniel Tarlow, and Tom Minka. A* sampling, 2015.

[17] Daniel Merchán, Jatin Arora, Julian Pachon, Karthik Konduri, Matthias Winkenbach, Steven Parks, and Joseph Noszek. 2021 amazon last mile routing research challenge: Data set. *Transportation Science*, 58(1):8–11, 2022.

[18] Shayan Oveis Gharan. Lecture 4–5: Effective resistance and simple random walks. Lecture notes, *Recent Advances in Approximation Algorithms*, University of Washington, 2015.

[19] Alessandro Panconesi and Aravind Srinivasan. Randomized distributed edge coloring via an extension of the chernoff–hoeffding bounds. *SIAM Journal on Computing*, 26(2):350–368, 1997.

[20] Prabhakar Raghavan and D Clark Tompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 1987.

[21] Ola Svensson, Jakub Tarnawski, and László A. Végh. A constant-factor approximation algorithm for the asymmetric traveling salesman problem. *Journal of the ACM*, 67(6), 2020.

[22] Vera Traub and Jens Vygen. An improved approximation algorithm for atsp. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, 2020.

[23] Vera Traub and Jens Vygen. *Approximation Algorithms for Traveling Salesman Problems*. Cambridge University Press, 2024.

[24] Vijay V. Vazirani. *Approximation Algorithms*. Springer Berlin, Heidelberg, 2001.